

Feature Importance Analysis

/Model_Comparison_and_Feature_Importance /

and

/Model_Comparison_Code/ (in the Michael branch)

In this portion, I had shared responsibilities with a great student, Michael Tao. I will be reporting mainly the portions that I was doing as well as what the code is doing as a whole. The main issues I dealt with in this project were just debugging code as well as trying to run the cloud machines.

The code (for both directories) is aimed to train, evaluate, compare various models. It will evaluate the various models based on configurations of input features, input-output steps, and feature importance. These models are known in the code as the `original_performance`.

The second half of the project is aimed at finding the best feature or the best combination of features. This is done by

1. removing one feature at a time to see how that changed the MSE, MAE, MAPE
2. having solo features with the label to see how each feature affects the MSE, MAE, MAPE
3. analyzing the top x features to find the best combinations of features.

The `model_comparison.py` script serves as the main file with the main function. It uses helper functions from `helpers.py` for preprocessing data, defining model architectures, and evaluating performance. The results, including metrics like MAE and MSE, are logged to CSV files.

/Model_Comparison_Code

This directory gives the most raw version of the code. You must follow the documentation on the readme in order to get results yourself. No results are stored here. Professor Teymourian recommended that we did this in order to save space on cloud machines when running.

Documentation on Results

stored in `saved_models` and `evaluation_results.csv`, `feature_importance_results.csv` and various model results.

evaluation_results_(shape).csv - main data that is stored

- the name of the csv changes based on the configuration that is being used.

- stores the results of having a single feature be evaluated with the label. The best model is always chosen but not stored. Then the script will run with the top x features until it reaches the maximum amount of features.
 - only stores MAE since MAE is used to find the best features
- incremental evaluation - based off of the ranking of the best features by MAE, we add these features one at a time to the model to evaluate how it performs.
- MAE, MAPE, MSE are stored for each model when evaluation type is the incremental type.
- the goal of this is to find the best feature when used solo. Then find the best group of features that work well together.
- MAE_diff, MSE_diff, MAPE_diff are stored to compare how the model did when it had all the features. Example below: $\text{MAE_diff} = \text{MAE_current} - \text{original_performance}$

original_performance was how the model performed with all features. Therefore, the lower and more negative the MAE_diff is, the better that combination of features works.

feature_importance_results_(shape).csv

- the name of the csv changes based on the configuration that is being used.
- stores results of the models after getting rid of one feature at a time
- stores MSE, MAE, MAPE for each time

Creating an Attention Mechanism Model

/AttentionMechanism/

This directory implements part of attention mechanism/ transformer. It begins with an implementation of the feature importance analysis. It also contains some jupyter notebooks on Attention Mechanism inspired by another implementation here - <https://dlsyscourse.org/lectures/>.

Implemented a simple self-attention model as well as a transformer model and saw how it performed in 365 days of data of station1 dataset. It performs pretty well.

The process of creating this model took a while because I was new to machine learning. I had to do a lot of research and watching of youtube videos to understand this attention mechanism. Here are some resources that helped a lot.

- <https://dlsyscourse.org/lectures/> - recommended by Dr. Teymourian. Transformers lectures helped a lot

- <https://www.youtube.com/watch?v=eMlx5fFNoYc> - 3blue1brown attention in transformers, visually explained
- <https://www.youtube.com/watch?v=ErnWZxJovaM> - intro to deep learning MIT
- Looking through the entire repository(tx-soil-moisture) and understanding it to find relevant code to help you

TRANSFORMER MODEL

AttentionMechanism/notebook/transformer.ipynb

Inspired from -

(https://github.com/dlsyscourse/public_notebooks/blob/main/rnn_implementation.ipynb)

The model has an initial linear layer (input_proj) that projects to the hidden dimension (hidden_dim).

A TransformerEncoderLayer with multi-head attention (n_heads) and feed-forward dimensions (dim_feedforward). The layer uses causal masking to prevent future information from leaking into past predictions as shown in the tutorial above.

Output Layer: The output_layer maps the transformed output to the target dimension (however many hours), predicting a single value per sample.

Training:

- MSE is used as the loss function
- MAE, MAPE, MSE are all calculated for each batch and averaged at the end

Validation:

- Evaluation occurs at the end of the epoch to see the validation loss

Testing:

- Average MSE, MAE, MAPE are all averaged

Graphs:

- Prediction vs. Actual. Sees how well the model performs. It compares the model predictions with actual target values
- training loss vs validation loss: sees if we're overfitting if the lines diverge. They don't

SELF-ATTENTION MODEL

AttentionMechanism/notebook/selfattention.ipynb

Inspired from -

(https://github.com/dlsyscourse/public_notebooks/blob/main/rnn_implementation.ipynb)

The model has an initial linear layer (input_proj) that projects to the hidden dimension (hidden_dim).

This transformation above prepares the input for the self-attention mechanism, implemented via nn.MultiheadAttention. This attention mechanism allows the model to

selectively focus on different parts of the time sequence, enabling it to capture dependencies and relationships within the sequence - here's a video that helped me understand it better.

- <https://www.youtube.com/watch?v=IFKRf-BAqZo&feature=youtu.be>

The multi-head attention layer also contains causal masking to ensure that the model cannot access future time steps to influence its decision

Output Layer: The output_layer maps the transformed output to the target dimension (however many hours), predicting a single value per sample.

Training:

- MSE is used as the loss function
- MAE, MAPE, MSE are all calculated for each batch and averaged at the end

Validation:

- Evaluation occurs at the end of the epoch to see the validation loss

Testing:

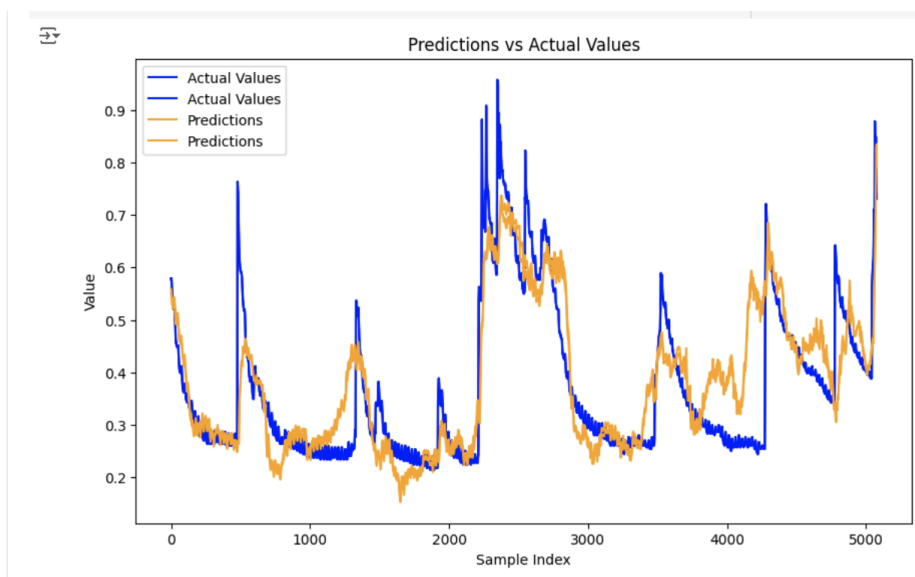
- Average MSE, MAE, MAPE are all averaged

Graphs:

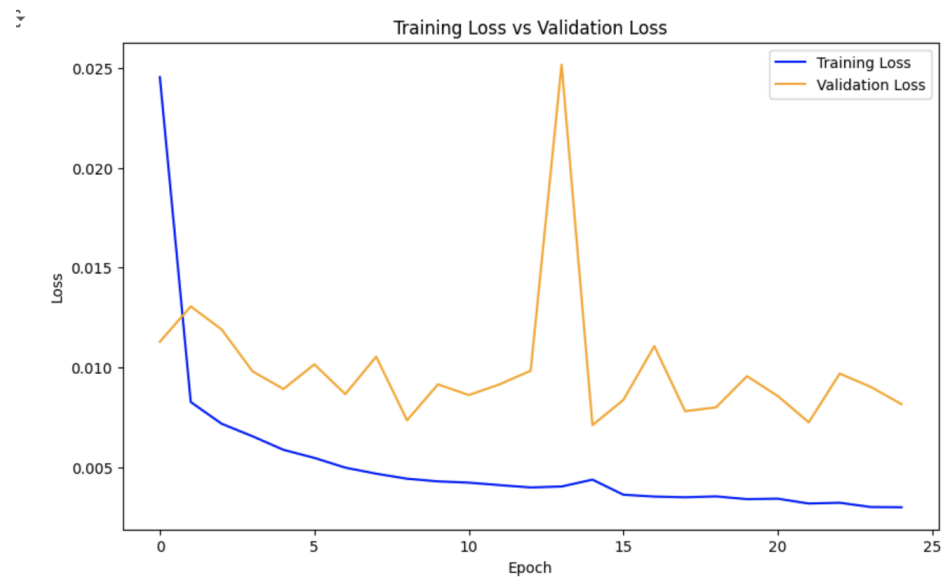
- Prediction vs. Actual. Sees how well the model performs. It compares the model predictions with actual target values
- training loss vs validation loss: sees if we're overfitting if the lines diverge. They don't

Results of transformer

AttentionMechanism/notebook/transformer.ipynb



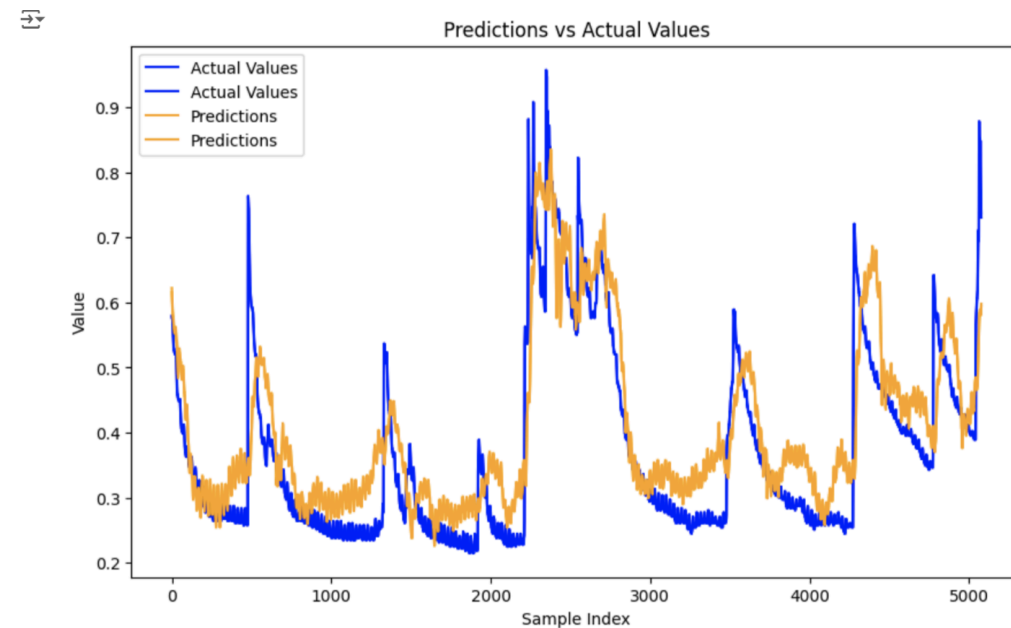
Training loss vs validation loss to see if overfitting



Test Loss (MSE): 0.007317
Average MSE: 0.007317
Average MAE: 0.056660
Average MAPE: 16.42%

Results of self-attention

AttentionMechanism/notebook/selfattention.ipynb



Training loss vs validation loss to see if overfitting

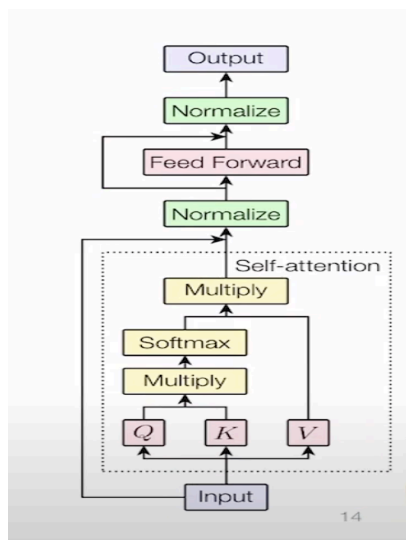


Test Loss (MSE): 0.006180
 Average MSE: 0.006180
 Average MAE: 0.062147
 Average MAPE: 17.86%

Inputs used

Window input - 168

Window output - 2



Above is the difference between a self-attention model and a transformer. The entire process is the transformer which contains a few more steps. The boxed portion is the self-attention portion

LSTM and BiLSTM Model with ATTENTION MODEL

AttentionMechanism/notebook/LSTMAttention.ipynb

Inspired from previous lstm and bi-lstm models that are in the repository.

LSTM with Attention

The model has two LSTM layers, the SeqSelfAttention layer (which focuses on the most relevant parts of the input sequence for the prediction task). It additionally has a Dense layer with 32 units and reLU activation and one final Dense layer with 1 unit and tanh activation for the prediction.

BiLSTM with Attention

The model has two biLSTM layers, the SeqSelfAttention layer (which focuses on the most relevant parts of the input sequence for the prediction task). It additionally has a Dense layer with 32 units and reLU activation and one final Dense layer with 1 unit and tanh activation for the prediction.

General notes for Training and Evaluation

Loss Function:

- Mean Squared Error (MSE) is the primary loss function.

Metrics:

- MAE, MAPE, and MSE are computed for all models to provide a well-rounded view of their performance.

Validation:

- Validation loss is monitored to prevent overfitting and guide early stopping.

Testing:

- The test set provides an unbiased evaluation of the model's final performance.

Graphs:

- Prediction vs. Actual: Visualizes the model's forecasting accuracy.

Training vs. Validation Loss: Ensures that the model generalizes well by comparing training and validation performance.